

lab07

April 22, 2026

0.1 Power spectral density using Welch's method for one-sided spectrum

1. Split signal into overlapping segments of length NFFT
2. Detrend each segment
3. Apply window (length NFFT!)
4. Zero-pad ($\geq$ NFFT, power of 2)
5. Compute FFT
6. Keep only positive frequencies (one-sided spectrum)
7. Apply proper normalization (window + Fs + one-sided scaling)

```
[31]: #-----
from pylab import *
import numpy as np
import types

#-----
def apply_window(x, window):
    """
    Apply a window to signal x.

    'window' can be:
    1. A function: window(x) returns a windowed version of x
    2. An array: same shape as x, applied multiplicatively

    In DSP terms:
    x_w[n] = x[n] * w[n]
    """
    if type(window) == types.FunctionType:
        # Window is a function that internally applies the weighting
        return window(x)
    else:
        # Window is an array + pointwise multiplication
        return window * x

#-----
def _spectral_helper(x, NFFT=256, Fs=2, detrend=detrend_none,
                    window=window_hanning, noverlap=0, pad_to=None):
```

```

"""
Compute windowed, segmented FFT slices of a signal.

This is a low-level routine used in spectral estimation (e.g. Welch PSD).

Pipeline:
1. Split signal into overlapping segments of length NFFT
2. Detrend each segment
3. Apply window (length NFFT!)
4. Zero-pad to length pad_to (>= NFFT, power of 2)
5. Compute FFT
6. Keep only positive frequencies (one-sided spectrum)
7. Apply proper normalization (window + Fs + one-sided scaling)

Returns:
- FFTSlices: normalized complex spectra (per segment)
- freqs: frequency axis (0 → Fs/2)
"""

# Original signal length
n = len(x)

#-----
# Ensure we have at least one full segment
#-----
if len(x) < NFFT:
    # Resize to NFFT and zero-pad the rest
    x = np.resize(x, (NFFT,))
    x[n:] = 0
    # NOTE: np.resize may repeat data, but we overwrite with zeros

#-----
# FFT length selection
#-----
if pad_to is None:
    pad_to = NFFT

# Force FFT length to next power of 2 (computational efficiency)
pad_to = 2**int(np.ceil(np.log2(pad_to)))

# Number of frequencies in one-sided spectrum
# (DC → Nyquist)
numFreqs = pad_to // 2 + 1

#-----
# Segment indexing (overlapping windows)

```

```

#-----
# Step size = hop = NFFT - noverlap
ind = list(range(0, n - NFFT + 1, NFFT - noverlap))

# Number of segments
numSlices = len(ind)

# Allocate array:
# rows → time segments
# cols → frequency bins
FFTSlices = np.zeros((numSlices, numFreqs), dtype=np.complex_)

# Simple list of indices for looping
slices = list(range(numSlices))

#-----
# Window normalization (VERY IMPORTANT)
#-----
# Apply window to a constant signal of ones → yields window itself
# Then compute L2 norm:
# normVal = sqrt(sum w[n]^2)
#
# This ensures energy normalization:
# compensates for attenuation introduced by the window
normVal = np.linalg.norm(apply_window(np.ones(NFFT), window))

#-----
# Main loop: process each segment
#-----
for iSlice in slices:

    # Extract segment of length NFFT
    segment = x[ind[iSlice]:ind[iSlice] + NFFT]

    # Remove mean / trend if needed
    segment = detrend(segment)

    # Apply window (length NFFT!)
    segment = apply_window(segment, window)

    # Compute FFT:
    # - zero-padded to length pad_to
    # - gives finer frequency grid (interpolation), not higher resolution
    spectrum = np.fft.fft(segment, n=pad_to)

    # Keep only positive frequencies (one-sided spectrum)
    FFTSlices[iSlice, :] = spectrum[:numFreqs]

```

```

#-----
# Frequency axis
#-----
# f_k = k * Fs / pad_to
freqs = float(Fs) / pad_to * np.arange(numFreqs)

#-----
# One-sided spectrum correction
#-----
# We removed negative frequencies.
# For real signals:
#  $|X(-f)|^2 = |X(f)|^2$ 
#
# To preserve total energy:
# double the power of interior bins
# → multiply amplitudes by sqrt(2)
#
# DO NOT scale:
# - DC (index 0)
# - Nyquist (last index)
FFTSlices[:, 1:-1] *= np.sqrt(2)

#-----
# Final normalization
#-----
# Divide by:
# 1. normVal → window energy normalization
# 2. sqrt(Fs) → ensures PSD units ( $V^2 / \text{Hz}$  after squaring)
#
# After this:
# |FFTSlices|^2 behaves like a properly scaled PSD contribution
return FFTSlices / normVal / np.sqrt(Fs), freqs

#-----
# NOTE (commented original line):
# Pxx = mean(|Slices|^2)
#
# This function returns the building blocks for PSD:
# averaging over slices is done outside
#-----

#-----
def psd(x, NFFT=256, Fs=2, detrend=detrend_none, window=window_hanning,
    ↪noverlap=0, pad_to=None):
    X, f = _spectral_helper(x, NFFT, Fs, detrend, window, noverlap, pad_to)
    Pxx = np.abs(X)**2

```

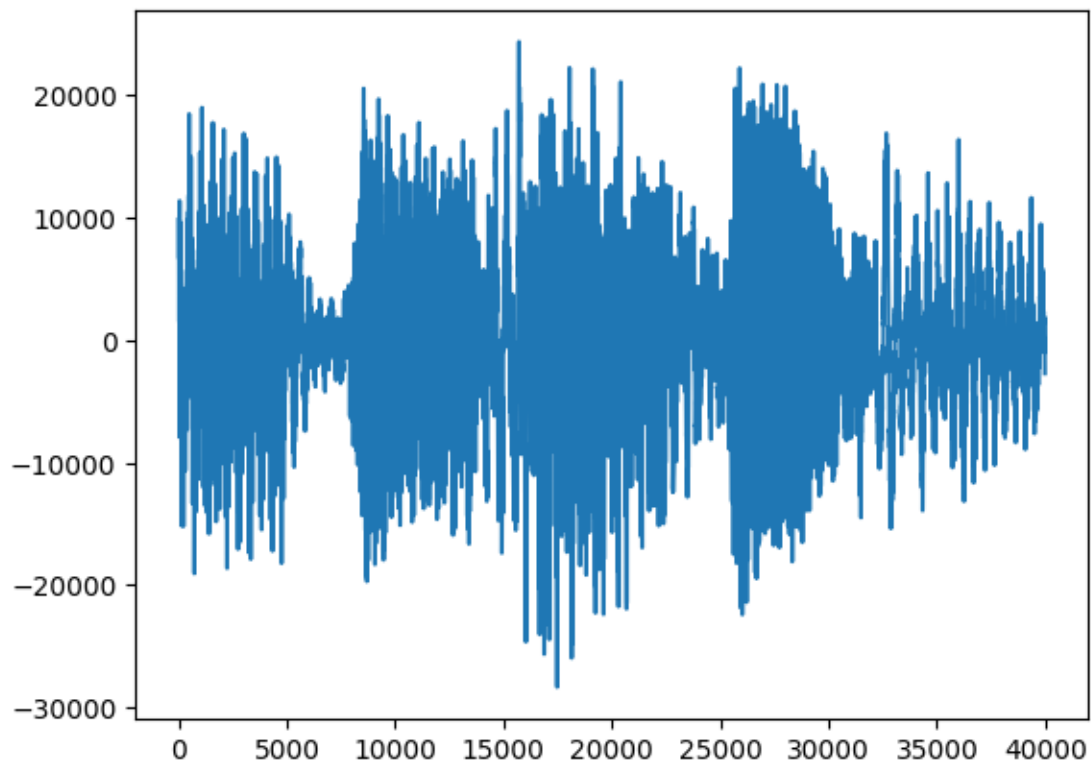
```
return Pxx.mean(axis=0), f
```

```
[32]: from scipy.io import wavfile
import scipy.io
samplerate, data = wavfile.read("02. School Boy-9.wav")
print(samplerate, data.shape)
```

44100 (882000, 2)

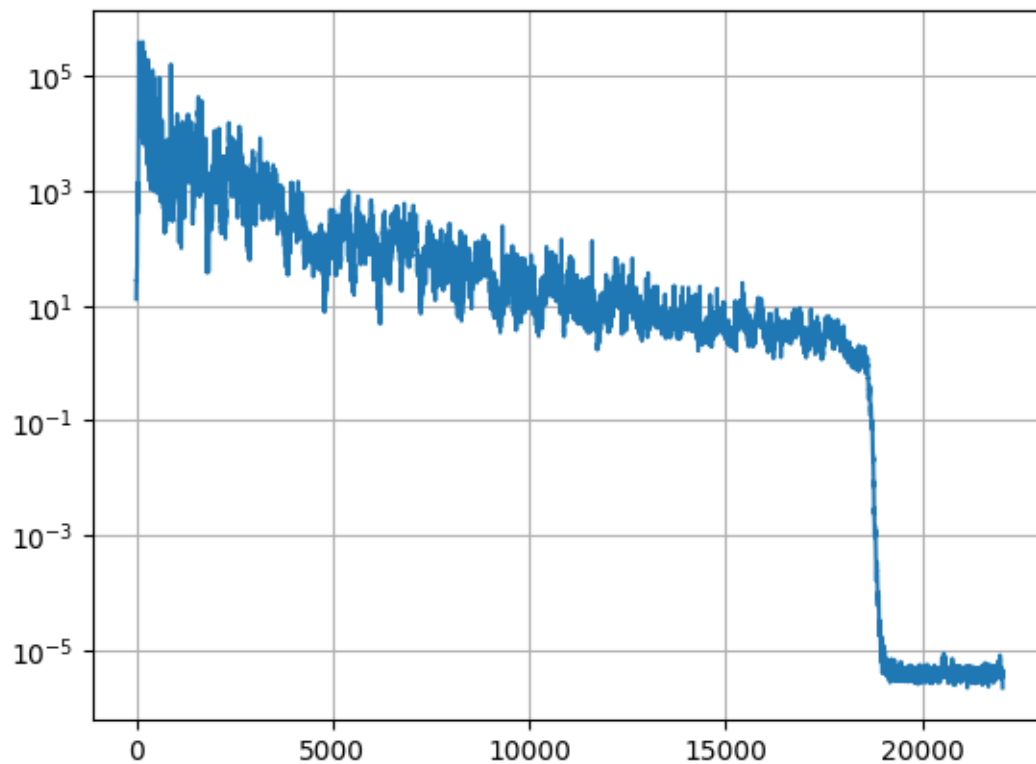
```
[33]: plot(data[10000:50000, 0])
```

[33]: [matplotlib.lines.Line2D at 0x700f3b24f9e0]



```
[34]: s, f = psd(data[:, 0], Fs=samplerate, NFFT=2**15, noverlap=2**14,
↳ detrend=detrend_none)
figure(1)
semilogy(f, s)
#xlim([0, 10])
#ylim([0, 100])
grid()
s
```

```
[34]: array([1.31055091e+01, 2.39932355e+01, 2.48665133e+01, ...,
          3.85225117e-06, 4.27725733e-06, 2.22841034e-06])
```



```
[35]: len(f) # NFFT//2 + 1
```

```
[35]: 16385
```

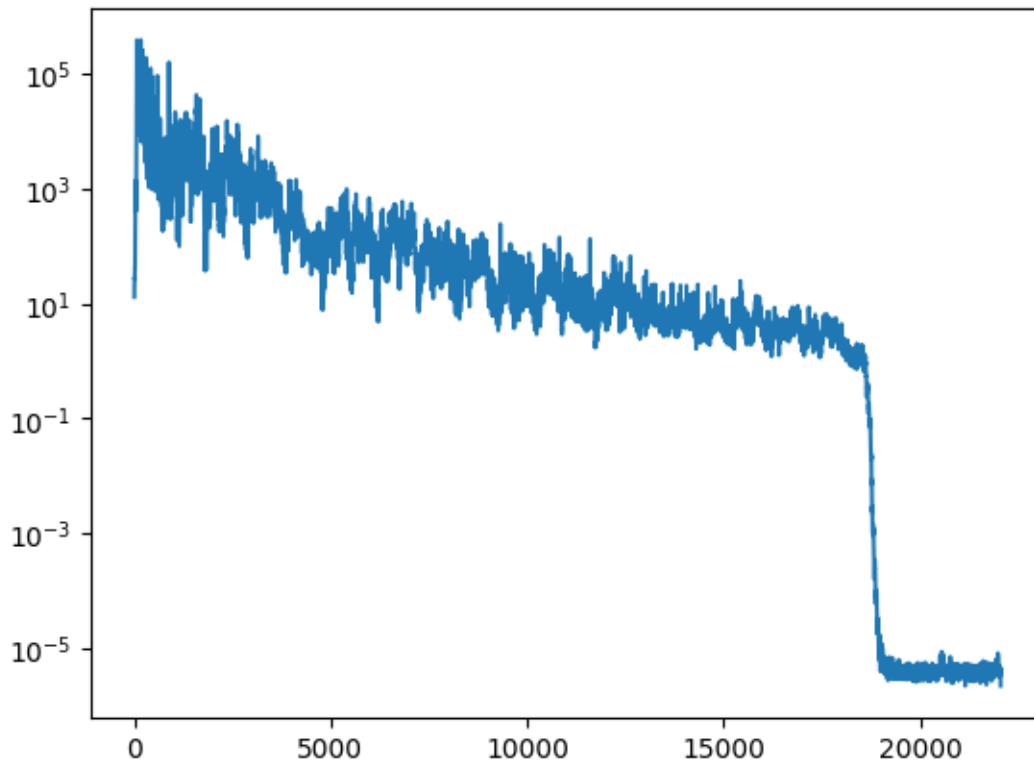
```
[36]: from scipy.signal import welch

f, s = welch(
    data[:, 0],
    fs=samplerate,
    window='hann',
    nperseg=2**15,
    noverlap=2**14,
    detrend=False,
    nfft=2**15,           # matches pad_to = NFFT
    return_onesided=True,
    scaling='density'     # ensures V^2/Hz
)
s
```

```
[36]: array([1.3105723e+01, 2.3993275e+01, 2.4866520e+01, ..., 3.8519356e-06,
          4.2773772e-06, 2.2273782e-06], dtype=float32)
```

```
[37]: semilogy(f, s)
```

```
[37]: [<matplotlib.lines.Line2D at 0x700f3d1b9880>]
```



```
[ ]:
```

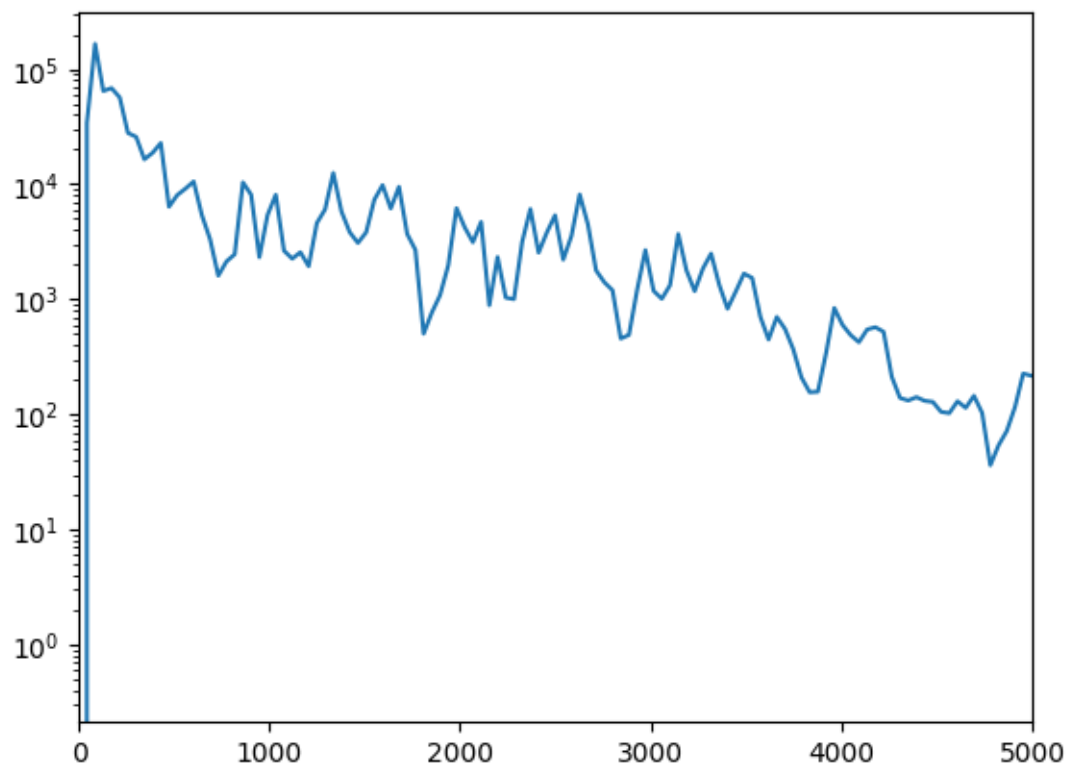
```
[38]: freq, Pxx = signal.welch(data[:, 0], fs=samplerate, nperseg=2**10,
    ↪noverlap=2**9, window='box', detrend='constant')
semilogy(freq, Pxx)
#semilogy(f, s)
xlim([0, 5000])
print(s, Pxx)
print(len(s), len(Pxx))
```

```
[1.3105723e+01 2.3993275e+01 2.4866520e+01 ... 3.8519356e-06 4.2773772e-06
2.2273782e-06] [0.00000000e+00 3.40120625e+04 1.65266297e+05 6.44871914e+04
6.82736094e+04 5.66429102e+04 2.78621816e+04 2.57531367e+04
1.64401758e+04 1.87071465e+04 2.27599531e+04 6.37076758e+03
8.03988135e+03 9.14491797e+03 1.05469414e+04 5.38832324e+03
3.32532275e+03 1.60508289e+03 2.13451636e+03 2.46232666e+03]
```

1.03656084e+04 8.09779932e+03 2.32309741e+03 5.40569092e+03
 8.08889844e+03 2.64515112e+03 2.25424414e+03 2.56276294e+03
 1.93995874e+03 4.59573486e+03 6.03193311e+03 1.25195801e+04
 5.78519775e+03 3.83268823e+03 3.08722339e+03 3.81550903e+03
 7.33067236e+03 9.81129590e+03 6.15482861e+03 9.49530371e+03
 3.69297461e+03 2.68776709e+03 5.03021759e+02 7.68125427e+02
 1.08727197e+03 1.97176282e+03 6.19230908e+03 4.22030762e+03
 3.13241431e+03 4.70174805e+03 8.88579041e+02 2.33254785e+03
 1.03463489e+03 1.00321356e+03 3.15739307e+03 6.09360645e+03
 2.54323755e+03 3.80342041e+03 5.37019385e+03 2.21136206e+03
 3.55747876e+03 8.14281836e+03 4.48496094e+03 1.78132874e+03
 1.40760083e+03 1.19824854e+03 4.56296295e+02 4.91287262e+02
 1.21223535e+03 2.67776880e+03 1.18468225e+03 1.01068268e+03
 1.32608850e+03 3.69743213e+03 1.78535754e+03 1.17907043e+03
 1.84882288e+03 2.49710059e+03 1.33157471e+03 8.29208252e+02
 1.16699329e+03 1.67223914e+03 1.53367358e+03 7.04549255e+02
 4.47787537e+02 7.02740967e+02 5.53446045e+02 3.69306458e+02
 2.09850525e+02 1.56080719e+02 1.57742813e+02 3.41693756e+02
 8.41341614e+02 5.99619873e+02 4.88744049e+02 4.24237061e+02
 5.47336121e+02 5.72559998e+02 5.23076721e+02 2.12407883e+02
 1.39226578e+02 1.32251205e+02 1.41425949e+02 1.31445847e+02
 1.28632141e+02 1.05451035e+02 1.02699524e+02 1.30279907e+02
 1.14258743e+02 1.45215347e+02 1.03197151e+02 3.62625885e+01
 5.40292892e+01 7.19258652e+01 1.16039452e+02 2.26566544e+02
 2.16999359e+02 2.03434708e+02 1.30373825e+02 1.98593002e+02
 1.58965851e+02 1.45845200e+02 3.73681580e+02 4.57333344e+02
 4.92356110e+02 2.93276703e+02 1.93840515e+02 1.06200302e+02
 5.06940193e+01 6.79688644e+01 2.04891937e+02 3.56130554e+02
 2.66116241e+02 1.70203110e+02 1.96845505e+02 1.15040909e+02
 1.44007141e+02 1.79043854e+02 2.21762253e+02 3.73651642e+02
 1.76966736e+02 1.02747070e+02 8.19675980e+01 3.09555511e+01
 2.75723438e+01 8.25430450e+01 1.85119644e+02 1.95100159e+02
 8.92792740e+01 9.95012360e+01 1.23243301e+02 2.10176071e+02
 2.05193970e+02 3.01150787e+02 3.03731750e+02 2.19025864e+02
 1.18660065e+02 6.90215149e+01 1.68841522e+02 1.93557983e+02
 1.97849655e+02 1.51717300e+02 2.28835205e+02 1.78634308e+02
 2.29736542e+02 2.34883789e+02 1.11215340e+02 6.92717056e+01
 3.56284103e+01 4.01230354e+01 7.32033081e+01 8.05351028e+01
 1.58784134e+02 2.20630157e+02 1.32632812e+02 9.89019165e+01
 6.22313499e+01 4.93365517e+01 6.54102631e+01 6.66346359e+01
 7.83108902e+01 6.84890594e+01 6.20176315e+01 7.76569214e+01
 8.29605331e+01 9.58144226e+01 9.01603394e+01 5.94514923e+01
 5.56477203e+01 3.22914619e+01 1.71532135e+01 2.51852760e+01
 2.45546894e+01 2.73012486e+01 6.94793243e+01 7.29942245e+01
 7.94653854e+01 4.03202400e+01 2.95899487e+01 4.34152794e+01
 6.26522369e+01 6.10022621e+01 9.64436646e+01 8.55093689e+01
 8.42602539e+01 6.92483749e+01 7.61915588e+01 9.31017838e+01
 9.20025787e+01 3.59180183e+01 2.45132866e+01 2.45940971e+01

1.53477039e+01	1.83616848e+01	1.56847172e+01	1.01444254e+01
2.56019573e+01	2.47080116e+01	3.08001480e+01	3.95430069e+01
3.94888382e+01	3.22075005e+01	3.83478355e+01	2.23543091e+01
1.60982456e+01	2.03154240e+01	1.47528715e+01	1.53610392e+01
1.96749058e+01	2.16192741e+01	2.44890537e+01	2.30393486e+01
2.66739540e+01	2.36179390e+01	2.18051300e+01	2.94575958e+01
1.30637007e+01	9.58464336e+00	1.01266747e+01	1.96539516e+01
1.45728807e+01	2.20178070e+01	3.53742790e+01	4.29067802e+01
5.07777328e+01	4.19088631e+01	4.10262108e+01	4.16130447e+01
3.65973091e+01	3.71313133e+01	3.00861797e+01	3.09158554e+01
1.53358727e+01	1.29815302e+01	1.73903236e+01	2.40213718e+01
3.90834045e+01	2.63335819e+01	1.85586777e+01	1.44176540e+01
1.39316282e+01	1.20504112e+01	1.39769459e+01	1.45983572e+01
1.53480825e+01	1.20332327e+01	1.65028038e+01	1.38313227e+01
9.92553139e+00	2.08453007e+01	1.60524101e+01	9.13942623e+00
7.04729414e+00	5.49720573e+00	8.86722565e+00	1.23159924e+01
1.22662325e+01	1.30949745e+01	1.45745974e+01	8.61306286e+00
7.88942719e+00	1.11248283e+01	1.67953300e+01	2.94658489e+01
2.46368408e+01	1.52453041e+01	1.74629002e+01	1.48358841e+01
1.61078110e+01	1.38823385e+01	1.21562166e+01	1.81765671e+01
1.85670643e+01	1.62861748e+01	1.18584833e+01	1.17975979e+01
8.97332096e+00	7.42559195e+00	9.10356998e+00	6.52654314e+00
7.49427557e+00	1.20671816e+01	1.02821932e+01	1.00545626e+01
1.15796165e+01	1.26510725e+01	1.46524801e+01	9.55427647e+00
9.00176334e+00	7.23673344e+00	6.68783855e+00	7.09868574e+00
7.44801044e+00	6.78012609e+00	6.55318022e+00	6.08817434e+00
7.43050289e+00	8.30080700e+00	9.84088898e+00	7.33183622e+00
5.73163652e+00	5.57518864e+00	7.52436924e+00	1.15472145e+01
9.14586830e+00	8.21786213e+00	9.31628323e+00	8.43142796e+00
8.30928993e+00	8.00703621e+00	7.80096817e+00	6.52971745e+00
5.36630392e+00	7.63519526e+00	5.00342846e+00	4.96821594e+00
4.74396420e+00	5.63375664e+00	5.55924702e+00	6.13217306e+00
4.59188461e+00	5.22947359e+00	6.60722971e+00	8.14011574e+00
8.17092133e+00	1.09350615e+01	1.18114500e+01	9.09543324e+00
5.54244137e+00	5.20320892e+00	4.50600338e+00	4.05269766e+00
4.42669010e+00	4.38863802e+00	5.02377272e+00	7.97494411e+00
1.12830219e+01	1.45457306e+01	1.41297340e+01	9.06245041e+00
8.42338085e+00	6.35006380e+00	4.74625874e+00	5.00590038e+00
5.04231691e+00	4.58466244e+00	6.15253162e+00	1.01000557e+01
1.31854258e+01	1.26612616e+01	9.52555561e+00	6.05812836e+00
4.60732746e+00	3.83459115e+00	5.00386763e+00	6.00482702e+00
4.10677862e+00	4.06949663e+00	5.65800238e+00	5.85401487e+00
4.97724247e+00	4.15692282e+00	4.88952684e+00	4.91000605e+00
4.99239683e+00	4.66184092e+00	5.62350273e+00	5.48732281e+00
6.91640997e+00	6.94956589e+00	5.09383106e+00	6.64512205e+00
7.03832436e+00	6.63261175e+00	4.21971798e+00	3.74730134e+00
3.19902730e+00	3.81798196e+00	3.97887516e+00	4.94165421e+00
4.67176008e+00	3.61685252e+00	4.43798542e+00	4.85290289e+00

3.82404065e+00 2.84894466e+00 3.36255145e+00 4.75762796e+00
6.18598318e+00 5.27287817e+00 4.82119226e+00 4.65659714e+00
3.79246449e+00 4.17862511e+00 5.43941164e+00 5.15309381e+00
3.60570049e+00 3.76863050e+00 4.06125784e+00 2.82653189e+00
3.10417271e+00 2.65583801e+00 2.56573009e+00 2.40567970e+00
2.26467872e+00 2.02326655e+00 2.01945496e+00 2.24453449e+00
2.12398672e+00 2.31572175e+00 2.44385958e+00 1.84720850e+00
1.82423520e+00 1.28577387e+00 1.10012031e+00 9.57709610e-01
8.94302309e-01 8.80334735e-01 8.74146581e-01 8.70224059e-01
8.67218018e-01 8.64518106e-01 8.62068117e-01 8.60395193e-01
8.58301282e-01 8.56408060e-01 8.54860246e-01 8.52961898e-01
8.51548731e-01 8.50156009e-01 8.48667562e-01 8.47351015e-01
8.45944762e-01 8.44781339e-01 8.43296528e-01 8.42213154e-01
8.41059208e-01 8.40046525e-01 8.38964760e-01 8.37973416e-01
8.36879611e-01 8.35972905e-01 8.34919810e-01 8.33930373e-01
8.33139658e-01 8.32104683e-01 8.31231892e-01 8.30397666e-01
8.29543650e-01 8.28727365e-01 8.28028202e-01 8.27195406e-01
8.26613426e-01 8.25821638e-01 8.25138628e-01 8.24576616e-01
8.23808789e-01 8.23182523e-01 8.22124541e-01 8.21811557e-01
8.21237087e-01 8.20754766e-01 8.19866002e-01 8.19757819e-01
8.19258988e-01 8.18736851e-01 8.18183541e-01 8.17743421e-01
8.17287862e-01 8.16976249e-01 8.16668153e-01 8.16295445e-01
8.15735877e-01 8.15555990e-01 8.15111041e-01 8.14827502e-01
8.14481735e-01 8.14313591e-01 8.14004064e-01 8.13857257e-01
8.13632667e-01 8.13419819e-01 8.13162148e-01 8.12865198e-01
8.12859893e-01 8.12692702e-01 8.12762678e-01 8.12544942e-01
8.12476575e-01 8.12736452e-01 8.12347412e-01 8.11912537e-01
4.06113863e-01]
16385 513



0.2 Parseval theorem

$$\int_{-\infty}^{\infty} |x(t)|^2 dt = \int_{-\infty}^{\infty} |X(f)|^2 df$$

$$\sum_{n=0}^{N-1} |x[n]|^2 = \frac{1}{N} \sum_{k=0}^{N-1} |X[k]|^2$$

```
[39]: ### Using FFT
import numpy as np

x = data[:, 0].astype(float)
N = len(x)

#-----
# Time-domain energy
#-----
E_time = np.sum(np.abs(x)**2)

#-----
# Frequency-domain energy
#-----
```

```

X = np.fft.fft(x)

E_freq = (1.0 / N) * np.sum(np.abs(X)**2)

E_time, E_freq

```

[39]: (30347298816264.0, 30347298816264.004)

```

[40]: ### Using power spectral density

import numpy as np

x = data[:, 0].astype(float)
N = len(x)

# -----
# Time-domain average power
# -----
P_time = np.mean(np.abs(x)**2)

# -----
# PSD from your routine
# no detrend, no window, no padding, no overlap
# -----
s, f = psd(
    x,
    Fs=samplerate,
    NFFT=N,
    noverlap=0,
    detrend=detrend_none,
    window=ones(N),
    pad_to=N
)

# -----
# Frequency-domain power from PSD
# -----
P_freq_sum = np.sum(s) * (f[1] - f[0])

P_time, P_freq_sum

```

[40]: (34407368.272408165, 34407368.27240818)

```

[41]: f, s = welch(
    x,
    fs=samplerate,
    window='boxcar',

```

```

    nperseg=N,
    noverlap=0,
    detrend=False,
    nfft=N,          # matches pad_to = NFFT
    return_onesided=True,
    scaling='density' # ensures  $V^2/\text{Hz}$ 
)
P_freq_sum = np.sum(s) * (f[1] - f[0])
P_time, P_freq_sum

```

[41]: (34407368.272408165, 34407368.27240818)

[]: